

02451 Introduction to Machine Learning

Week 6: Decision trees, overfitting and cross-validation

Bjørn Sand Jensen

10 March 2026

DTU Compute, Technical University of Denmark



Today

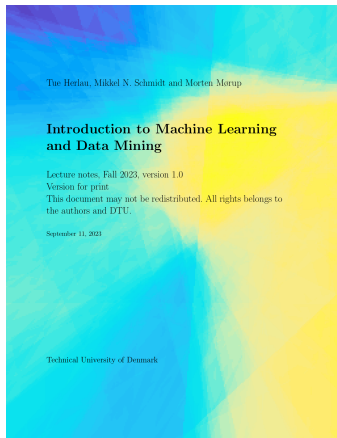
Feedback Groups of the day:

Adelin , Lucas Agerskov, Benjamin Philip Bakkelev, Christiane Louise Bang, Sophie Mary Carruthers, Kristina Carter, Inyoung Chae, Nadin Chehaiber, Hannah Erichsen, Silas Have Filtenborg, Sergi Fornós Zapater, Dag Lavard Gabriel Skotte Friis-Holm, Frederik Borrit Høgsbro, Gustav Valdemar Jensen, Andrejs Kocins, Lucie Kosatová, Jól Thomas Kristjánsson, Hayden Leung, Victor Baarup Lorenzen, Matias Haage Them Nielsen, Malthé Lucas Nørholm Nielsen, Anya Helle Pritzl, Mithra Iyengar Ramprabu, Francisco Javier Rodriguez, Lærke Knudsen Skaarup, Magnus Brandt Sløgedal, Tobias Christian Kragelund Strande, Wei Lin Ter, Mikkel Walz.

Reading/homework material:

Chapter 9,10, 12.3

P10.1, P10.2, P12.1



Lecture Schedule

- 1 Introduction, data, and visualization
3 February: C1, C2, C7
- 2 Summary statistics, similarity, and nearest neighbors
10 February: C4, C12
- 3 Computational linear algebra and PCA
17 February: C3
- 4 Probability theory, Bayes and Naive Bayes
24 February: C5, C6, C13 (Assignment 1-3 due on 26 February)
- 5 Bayesian inference and linear models
3 March: C5, C6, C8
- 6 Decision trees, overfitting and cross-validation
10 March: C9, C10
- 7 Performance evaluation and AUC
17 March: C11, C16 (Assignment 4-6 due on 19 March)
- 8 Artificial Neural Networks and Optimization
24 March: C15
- 9 Bias/Variance and ensemble methods
7 April: C14, 17
- 10 K-means and hierarchical clustering
14 April: C18 (Assignment 7-9 due on 16 April)
- 11 Mixture models and density estimation
21 April: C19, C20
- 12 Representation learning
28 April: Material provided separately
- 13 Recap and discussion of the exam
5 May: C1-C20 (Assignment 10-12 due on 7 May)

Online help: Piazza

Videos of lectures: <https://panopto.dtu.dk>

Streaming of lectures: Zoom (link on DTU Learn)

Practicalities and announcements

- Survey on Assignments 1-3
- sklearn vs basic numpy
- Videos
- **TA rooms have changed**

Recap lecture 5

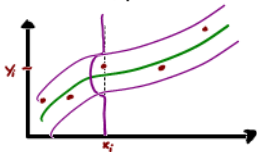
$$f(x; w) = w^T \tilde{x}$$

Regression w/ linear models

$$y_i = f(x_i; w) + \varepsilon_i \quad \varepsilon_i \sim \mathcal{N}(\varepsilon | 0, \sigma^2)$$

$$p(y_i | x_i; w) = \mathcal{N}(y_i | \mu = w^T x_i, \sigma^2)$$

$$\arg \max_w \prod_{i=1}^N p(y_i | x_i; w) \Rightarrow w^* = (X^T X)^{-1} X^T Y$$



$$f(x; w) = w_0 + w_1 x + w_2 x^3 + w_3 \sin(x) = w^T \tilde{x}$$

Bayesian inference

$$p(w | x, y) = \frac{p(y | x, w) p(w)}{p(y | x)}$$

$$\text{MAP} \quad \arg \max_w \prod_{i=1}^N p(y_i | x_i; w) p(w) \Rightarrow \arg \min_w - \sum_{i=1}^N \log p(y_i | x_i; w) - \log p(w)$$

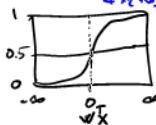
$$\text{Max Likelihood} \quad \arg \max_w \prod_{i=1}^N p(y_i | x_i; w) \Rightarrow \arg \min_w - \sum_{i=1}^N \log p(y_i | x_i; w)$$

(Binary) classification w/ linear models

$$p(y_i | x_i; w) = \sigma(w^T x_i) = \frac{1}{1 + e^{-w^T x_i}} \quad f(x; w) = w_0 + w_1 x_1 + w_2 x_2$$

$$\arg \max_w \prod_{i=1}^N p(y_i | x_i; w)$$

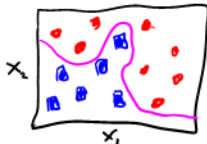
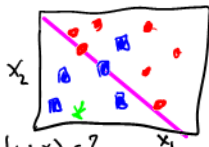
(w* not in closed-form)



Hard class (simple):

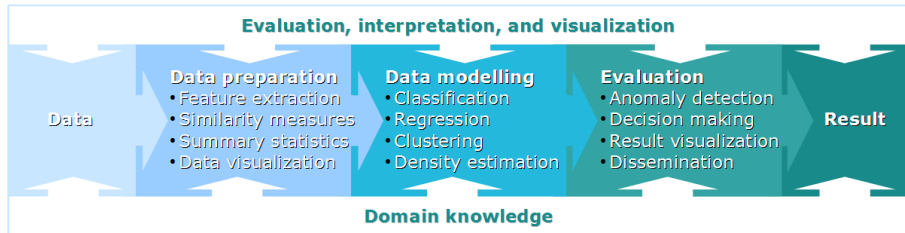
$$\hat{y} = \begin{cases} 1 & \text{if } p(y^* | x; w) \geq 0.5 \\ 0 & \text{if } p(y^* | x; w) < 0.5 \end{cases}$$

$$\begin{aligned} \text{red dot} &: y=0 \\ \text{blue square} &: y=1 \end{aligned}$$



$$p(y=1 | w, x) = ?$$

Learning Objectives



Learning Objectives

- Explain and apply decision tress
- Explain the difference between training, test and generalization error
- Explain how cross-validation can be used for (i) performance evaluation (ii) model selection
- Explain how cross-validation can be used in practical scenarios.

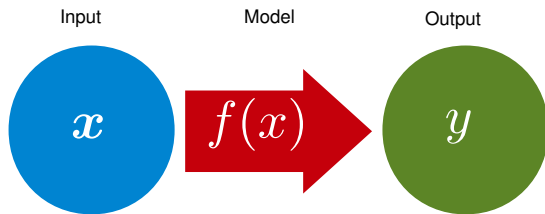
Roadmap

- Decision tress (and motivation for overfitting)
- Introduce errors
 - Training error
 - Test error
 - Generalization error
- Introduce cross-validation
 - **Basic cross validation** for **performance evaluation**
 - **Cross-validation** for **model selection**
 - **Two-level cross-validation** for **model selection and performance evaluation**

Supervised learning

Mapping between domains

- Classification: Discrete output
- Regression: Continuous output



Decision Trees

Decision trees

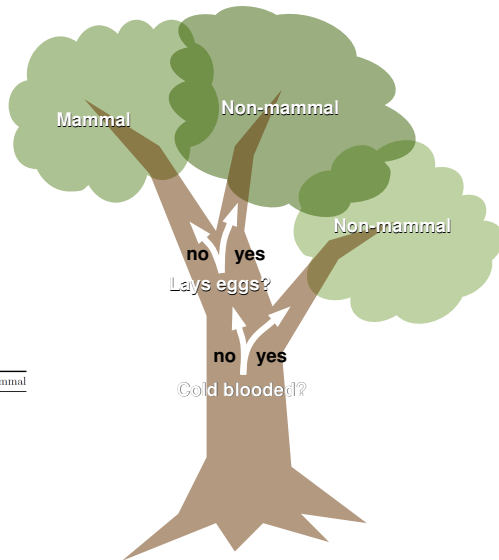
- Do you know the game “20 questions to the professor”?

Q1	Is it an Animal? Yes.
Q2	Can you hold it? No.
Q3	Does it live in groups (gregarious)? Yes.
Q4	Are there many different sorts of it? No.
Q5	Can it jump? Yes.
Q6	Does it eat seeds? No.
Q7	Is it white? Sometimes.
Q8	Is it black and white? No.
Q9	Does it have paws? Yes.
Q10	Can you see it in a zoo? Yes.
Q11	Does it roar? Yes.
Q12	Is it worth a lot of money? Yes.
Q13	Does it have spots? Yes.
Q14	Is it multicoloured? Yes.
Q15	Can you make money by selling it? Yes.
Q16	Does it live in the jungle? Yes.
Q17	I guessed that it was a leopard? Wrong.
Q18	Does it like to play? Yes.
Q19	I guessed that it was a cheetah? Wrong.
Q20	I am guessing that it is a siberian tiger? Correct.

Decision trees

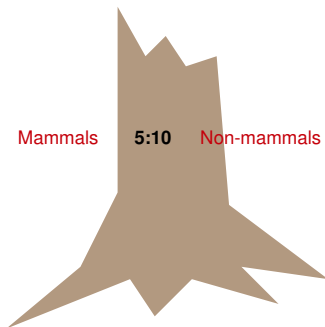
- Ask a serie of questions until a conclusion is reached
- **Example:** Classify **vertebrates** as
 - **Mammal** or
 - **Non-mammal**
- **Learning task:**
 - Which questions should we ask?

Name	x_1 : Cold Blooded	x_2 : Has Legs	x_3 : Lay Eggs	x_4 : Has Fur	y : Mammal
Snake	yes	-	yes	-	-
Starfish	yes	-	yes	-	-
Bluebird	-	yes	yes	-	-
Blackbird	-	yes	yes	-	-
Earthworm	yes	-	yes	-	-
Chameleon	yes	-	yes	-	-
Ant	yes	-	yes	-	-
Jellyfish	yes	-	yes	-	-
Snail	yes	-	yes	-	-
Sea Urchin	yes	-	yes	-	-
Dolphin	-	-	-	-	yes
Rat	-	yes	-	yes	yes
Dog	-	yes	-	yes	yes
Monkey	-	yes	-	yes	yes
Lion	-	yes	-	yes	yes



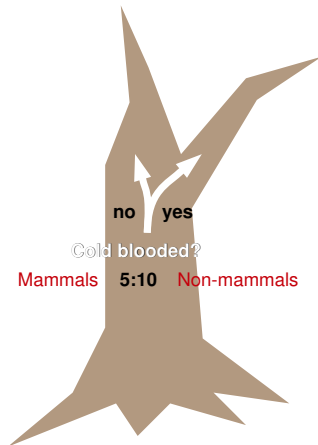
Hunt's algorithm

- Assign all data objects to the root



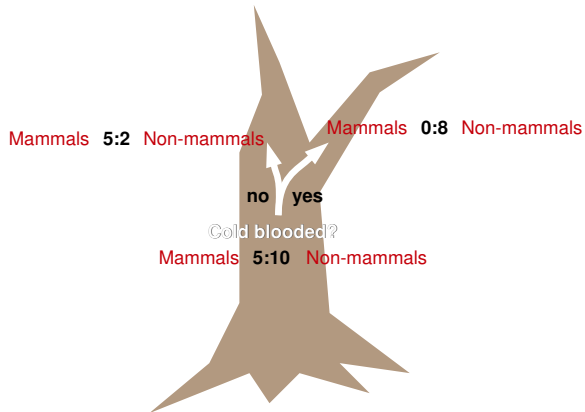
Hunt's algorithm

- Select an attribute test condition
 - Find a good question to ask



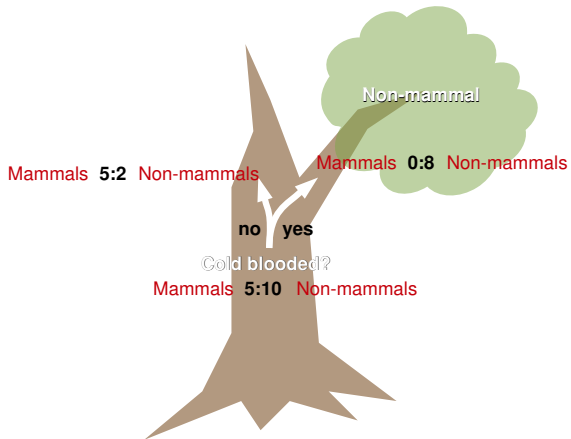
Hunt's algorithm

- Partition the data objects into subsets according to the test condition



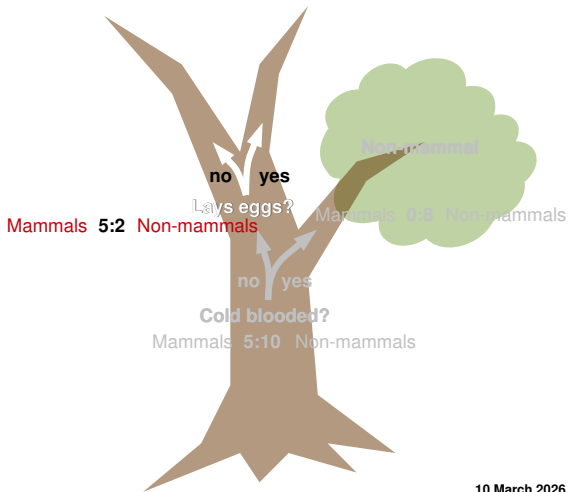
Hunt's algorithm

- If all data object belong to the same class
 - Create a leaf node



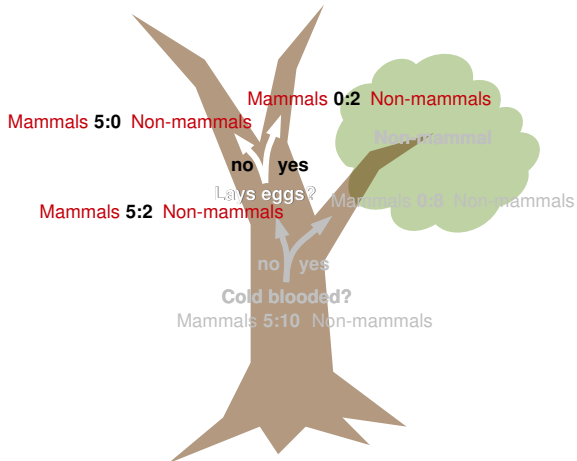
Hunt's algorithm

- Repeat for each non-leaf node:
 - Select an attribute test condition



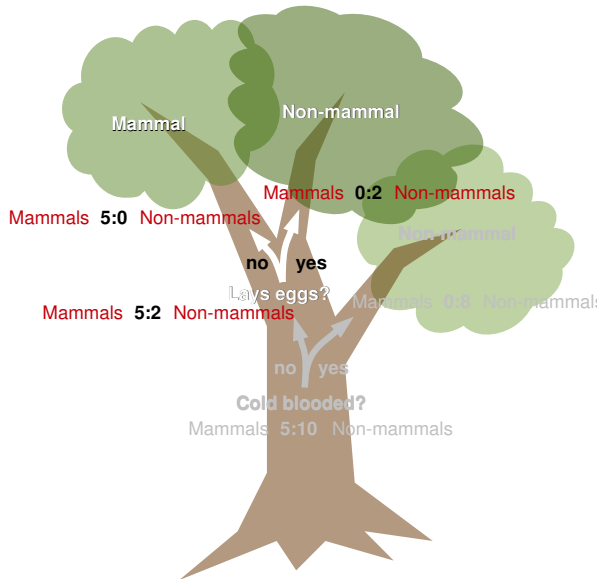
Hunt's algorithm

- Partition the data objects into subsets according to the test condition



Hunt's algorithm

- If all data object belong to the same class
 - Create a leaf node



Hunt's algorithm

- But how to we find the **best question** at each step?

Algorithm 2: Hunt's algorithm for decision trees

Require: Initial tree T only containing the root node

Require: D_r : Dataset associated with the current branch.

Initially just the full dataset

if The **stop criterion** is met **then**

Add a leaf node to the tree which assigns every observation to the most prevalent class in D_r .

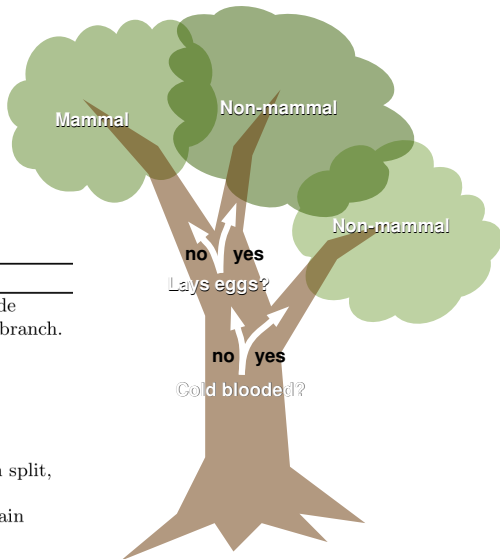
else

Try a number of different splits on D_r . For each split, compute the **purity gain** and select the split

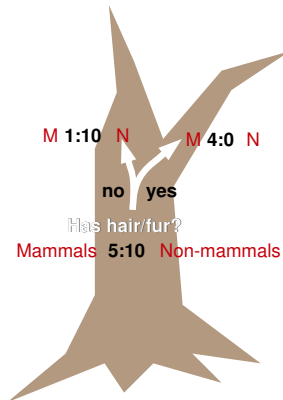
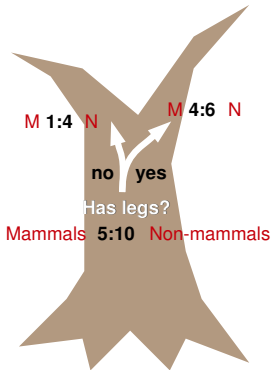
$D_r = \{D_{v_1}, \dots, D_{v_K}\}$ with the highest purity gain

Recursively call the method on $D_{v_1}, \dots, D_{v_K}$

end if



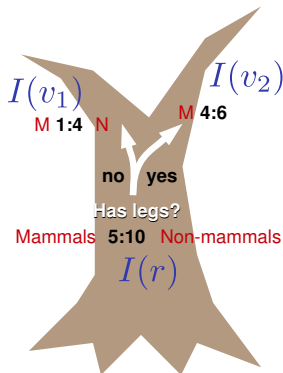
Which split is best?



Which split is best?

- Create a measure Δ (**the purity gain**) of how good a split is
- A binary split creates 3 partitions: the root r and the right/left branches v_1 , v_2 .
- For each partition, we compute $I(r)$, $I(v_1)$, $I(v_2)$ (the **impurity**)
- Purity gain is the **weighted reduction in impurity**:

$$\Delta = I(r) - \sum_{k=1}^{K=2} \frac{N(v_k)}{N(r)} I(v_k)$$



$N(r) = 15$,
 $N(v_1) = 5$,
 $N(v_2) = 10$

Which split is best?

- Create a measure Δ (**the purity gain**) of how good a split is
- A binary split creates 3 partitions: the root r and the right/left branches v_1 , v_2 .
- For each partition, we compute $I(r)$, $I(v_1)$, $I(v_2)$ (the **impurity**)
- Purity gain is the **weighted reduction in impurity**:

$$\Delta = I(r) - \sum_{k=1}^{K=2} \frac{N(v_k)}{N(r)} I(v_k)$$

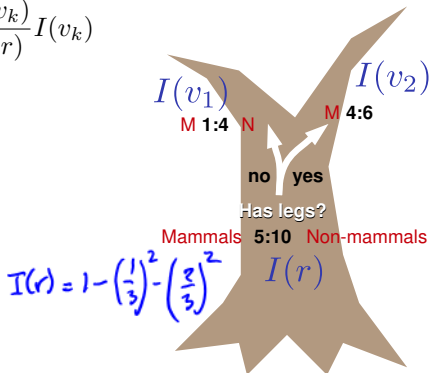
The impurity measure, $I()$, can be one of the following

$$\text{Entropy}(v) = - \sum_{c=1}^C p(c|v) \log_2 p(c|v),$$

$$\text{Gini}(v) = 1 - \sum_{c=1}^C p(c|v)^2,$$

$$\text{ClassError}(v) = 1 - \max_c p(c|v).$$

$$p(c|v) = \frac{\{\text{Nr. in class } c \text{ in branch } v\}}{N(v)}$$

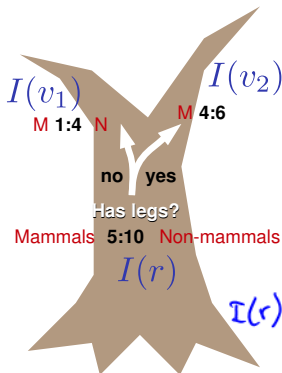


$N(r) = 15,$
 $N(v_1) = 5,$
 $N(v_2) = 10$

Quiz 3: Impurity gain

If we use the Gini index as impurity measure I , what is the purity gain Δ for the split indicated by the tree?

$$\Delta = I(r) - \sum_{k=1}^{K=2} \frac{N(v_k)}{N(r)} I(v_k)$$



$$\begin{aligned} N(r) &= 15, \\ N(v_1) &= 5, \\ N(v_2) &= 10 \end{aligned}$$

$$I(r) = 1 - \left(\frac{1}{3}\right)^2 - \left(\frac{2}{3}\right)^2$$

A. ≈ 0.0177

B. ≈ 0.104

C. ≈ 0.129

D. ≈ 0.2

E. Don't know.

The impurity measure, $I()$, can be one of the following

$$\text{Entropy}(v) = - \sum_{c=1}^C p(c|v) \log_2 p(c|v),$$

$$\text{Gini}(v) = 1 - \sum_{c=1}^C p(c|v)^2,$$

$$\text{ClassError}(v) = 1 - \max_c p(c|v).$$

$$p(c|v) = \frac{\{\text{Nr. in class } c \text{ in branch } v\}}{N(v)}$$

Quiz 1: Impurity gain—solution

We first identify $p(c|v)$:

$$p(\text{Mammal}|r) = 5/15 = 1/3$$

$$p(\text{Non-Mammal}|r) = 10/15 = 2/3$$

$$p(\text{Mammal}|v_1) = 1/5$$

$$p(\text{Non-Mammal}|v_1) = 4/5$$

$$p(\text{Mammal}|v_2) = 4/10$$

$$p(\text{Non-Mammal}|v_2) = 6/10$$

Using Gini impurity we get:

$$I(r) = 1 - \left(\frac{1}{3}\right)^2 - \left(\frac{2}{3}\right)^2 = \frac{4}{9} \quad (0.44)$$

$$I(v_1) = 1 - \left(\frac{1}{5}\right)^2 - \left(\frac{4}{5}\right)^2 = \frac{8}{25} \quad (0.32)$$

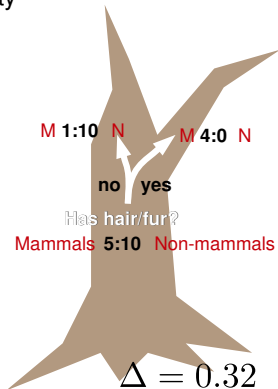
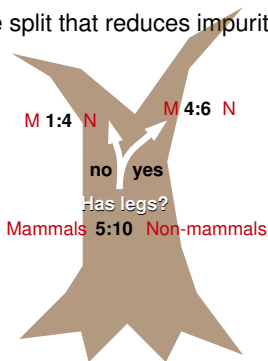
$$I(v_2) = 1 - \left(\frac{4}{10}\right)^2 - \left(\frac{6}{10}\right)^2 = \frac{48}{100} \quad (0.48)$$

and finally

$$\Delta = I(r) - \frac{5}{15}I(v_1) - \frac{10}{15}I(v_2) \approx 0.0177$$

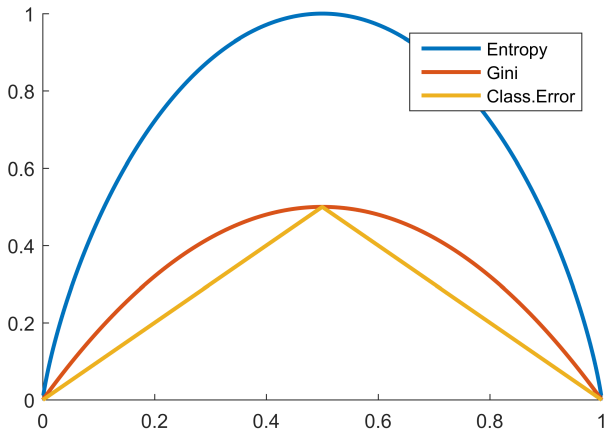
Selecting the best split

- Consider a large number of possible splits
- Compute a measure of impurity after the proposed split
- For each new branch of the tree
 - For each new branch of the tree
 - Compute weighted average impurity
- Choose split that reduces impurity most



Which impurity measure to use?

For a two class problem:



Types of splits to consider?

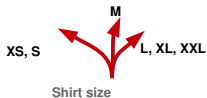
- Binary



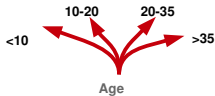
- Nominal



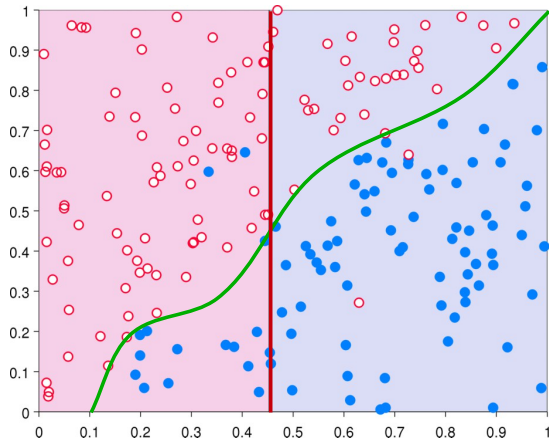
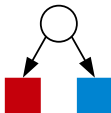
- Ordinal



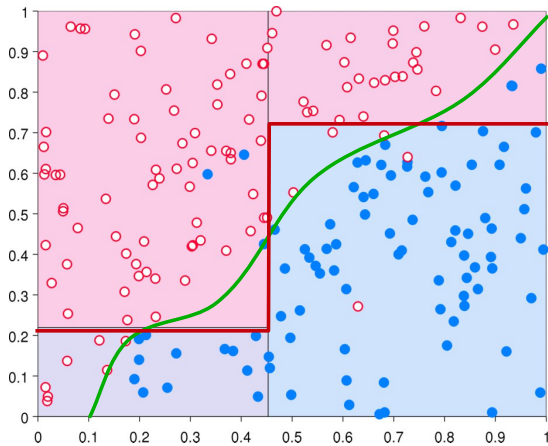
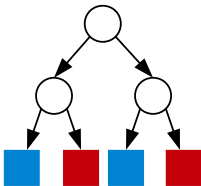
- Continuous



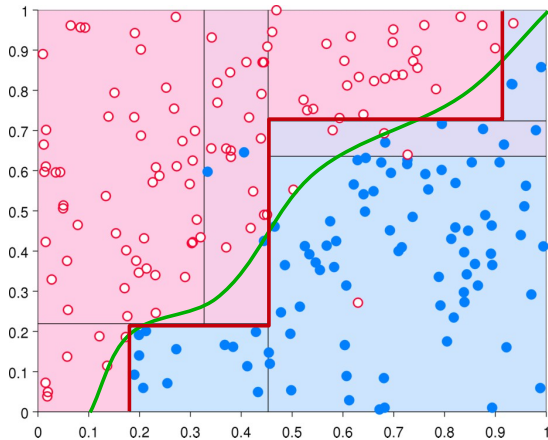
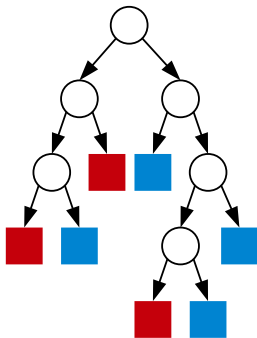
Classification tree



Classification tree



Classification tree

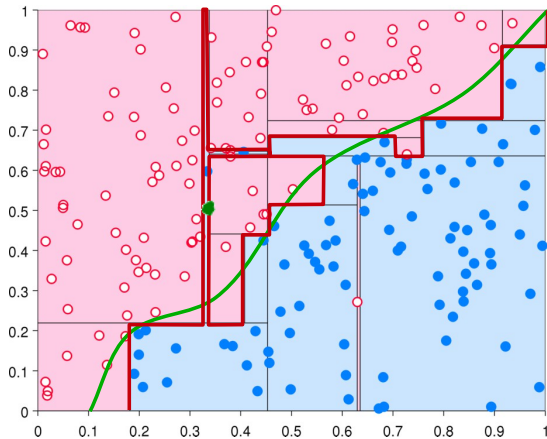
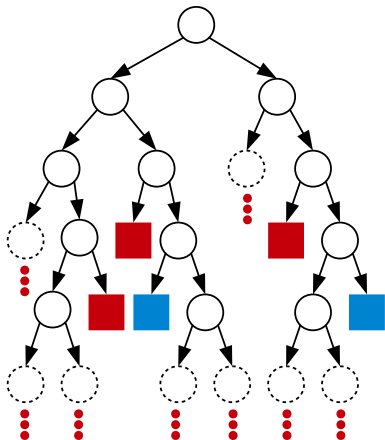


Classification tree

Common stopping criteria:

All points have the same class label.

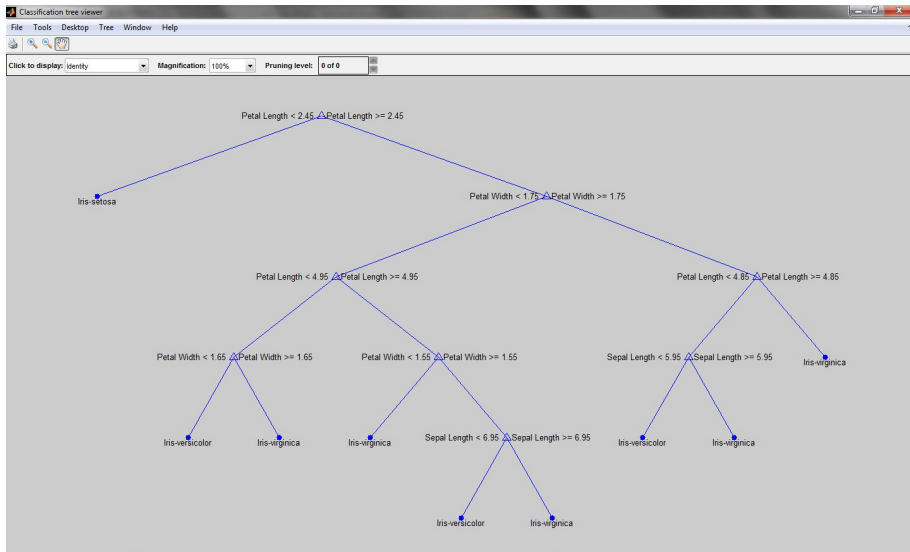
The number of points is below some minimum threshold.



Example: The Iris Dataset

- **Three types of Iris flowers**
 - 50 instances of each class, 150 in total
- **Attributes**
 - Sepal (outermost leaves)
 - length in cm
 - width in cm
 - Petal (innermost leaves)
 - length in cm
 - width in cm
 - Class of flower
 - Iris Setosa
 - Iris Versicolour
 - Iris Virginica

Fower ID	Attribute			
	Sepal Length	Sepal Width	Petal Length	Petal Width
1	5.1	3.5	1.4	0.2
2	4.9	3.0	1.4	0.2
3	4.7	3.2	1.3	0.2
4	4.6	3.1	1.5	0.2
.	.	.	.	.
.	.	.	.	.
150	5.9	3.0	5.1	1.8



What would the following
Iris flower be classified as?

Sepal Length	Sepal Width	Petal Length	Petal Width
4.0	3.5	3.0	2.0

Regression trees

Algorithm 4: Hunt's algorithm for regression trees

Require: Initial tree T only containing the root node

Require: D_r : Dataset associated with the current branch. Initially just the full dataset

if The **stop criterion** is met **then**

 Add a leaf node to the tree which assigns every observation the mean value of the nodes in D_r :

$$\hat{y}(r) = \frac{1}{N(r)} \sum_{i \in r} y_i$$

else

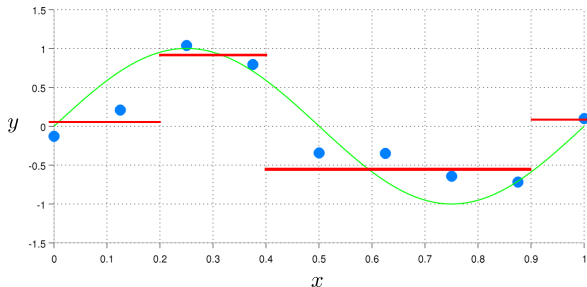
 Try a number of different splits on D_r . For each split, compute the **purity gain** using the sum-of-squares impurity measure and select the split $D_r = \{D_{v_1}, \dots, D_{v_K}\}$ with the highest purity gain

 Recursively call the method on $D_{v_1}, \dots, D_{v_K}$

end if

Use mean squared error as purity gain

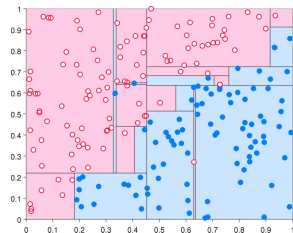
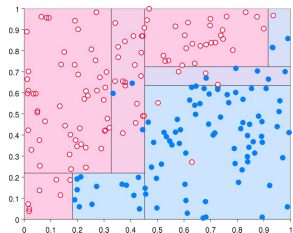
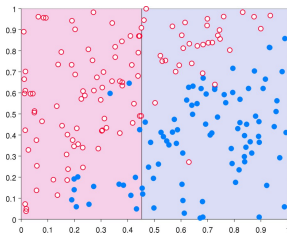
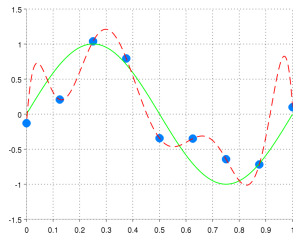
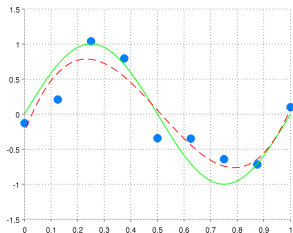
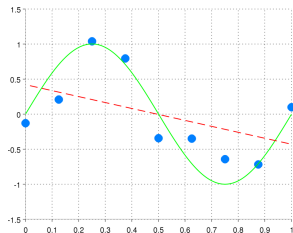
$$I(v) = \frac{1}{N(v)} \sum_{i \in v} (y_i - \hat{y}_v)^2, \quad \hat{y}_v = \frac{1}{N(v)} \sum_{i \in v} y_i$$



14:10

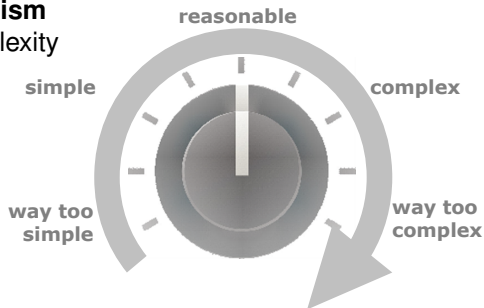
Overfitting and cross-validation

Model Overfitting - why multiple models?



Control the model complexity

- Find **parameter** or **mechanism** in model that controls complexity

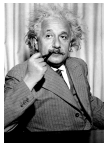


Lex Parsimoniae, Law of parsimony



Given two models with same predictive performance, the simpler model is preferred over the more complex model
- William of Ockham (1288-1347)
(paraphrased)

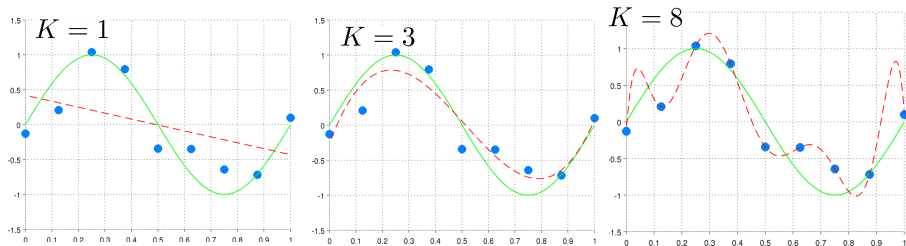
https://commons.wikimedia.org/wiki/File:William_of_Ockham.png



"Everything should be made as simple as possible, but not simpler" - Einstein

Linear regression

- Linear regression on non-linearly transformed inputs (polynomials)
$$f(x) = w_0 + w_1x + \dots + w_8x^8$$
- **Control complexity:** Choose a suitable value for K



Solution: Assess model performance correctly and select best model

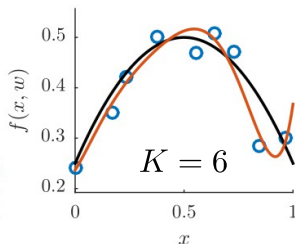
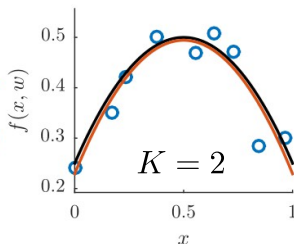
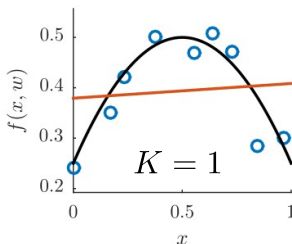
Training error

- Suppose we train 3 models on a dataset of 9 observations

$\mathcal{M}_1 = \{1\text{'st order polynomial}\}$

$\mathcal{M}_2 = \{2\text{'nd order polynomial}\}$

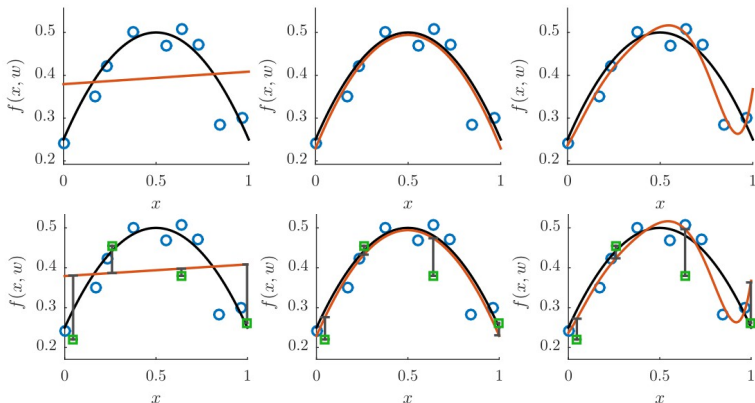
$\mathcal{M}_3 = \{6\text{'th order polynomial}\}$



$$E_{\mathcal{M}_k}^{\text{train}} = \frac{1}{N^{\text{train}}} \sum_{i \in \mathcal{D}^{\text{train}}} (y_i - f_{\mathcal{M}_k}(x_i, \mathbf{w}))^2.$$

Test error

- Test error is obtained by testing the trained models on **new** data



$$E_{\mathcal{M}_k}^{\text{train}} = \frac{1}{N^{\text{train}}} \sum_{i \in \mathcal{D}^{\text{train}}} (y_i - f_{\mathcal{M}_k}(x_i, \mathbf{w}))^2.$$
$$E_{\mathcal{M}_k}^{\text{test}} = \frac{1}{N^{\text{test}}} \sum_{i \in \mathcal{D}^{\text{test}}} (y_i - f_{\mathcal{M}_k}(x_i, \mathbf{w}))^2$$

Overfitting

- **Overfitting** is that the training error usually decreases for overly complex models while the test error increases.
- Test error is more representative of the true error.
- **Never, ever validate a model on the same data it was trained upon!**



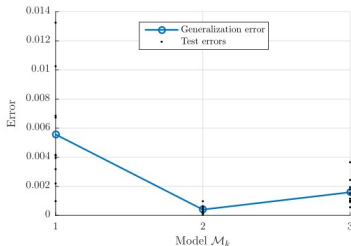
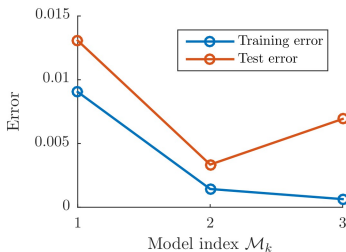
Generalization error

$$\int p(x, y) L(y, f_{\mathcal{M}}(x; \omega)) dx dy$$

- The generalization error is the test error evaluated over an infinitely large test set
- The generalization error is the "true performance" of the trained model
 - Train model $\mathcal{M}$ on the available dataset $\mathcal{D}$ to get prediction rule $f_{\mathcal{M}}$
 - Compute $E_{\mathcal{M}}^{\text{gen}} = \mathbb{E}_{(x, y)} [L(y, f_{\mathcal{M}}(x))]$
- If we somehow had many test sets $\mathcal{D}_1, \dots, \mathcal{D}_K$ we can make an estimate



$$E_{\mathcal{M}}^{\text{gen}} \approx \frac{1}{K} \sum_{k=1}^K E_{\mathcal{M}, \mathcal{D}_k}^{\text{test}}$$



Basic cross-validation

- **Purpose: Estimate the generalization error**

- **Holdout:** Partitions dataset in two (training, test), approximate the generalization error based on the resulting test set

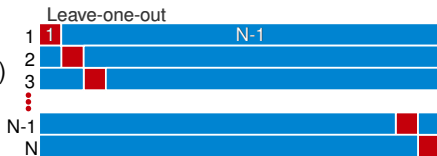
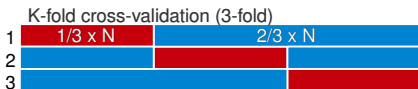
$$\mathcal{D} = \mathcal{D}^{\text{train}} \cup \mathcal{D}^{\text{test}}$$
$$E_{\mathcal{M}}^{\text{gen}} \approx E_{\mathcal{M}}^{\text{test}}$$

- **K-fold:** Partitions dataset in K parts. Each part is a test set and the other $K - 1$ training sets

$$\mathcal{D} = \mathcal{D}_1 \cup \mathcal{D}_2 \cup \dots \cup \mathcal{D}_K$$
$$E_{\mathcal{M}}^{\text{gen}} \approx \frac{1}{K} \sum_{k=1}^K E_{\mathcal{M},k}^{\text{test}}$$

- **Leave-one-out:** Partitions dataset into N parts. Let each observation be a test set and the other $N - 1$ training sets (K -fold with $K = N$)

$$\mathcal{D} = \mathcal{D}_1 \cup \mathcal{D}_2 \cup \dots \cup \mathcal{D}_N$$
$$E_{\mathcal{M}}^{\text{gen}} \approx \frac{1}{N} \sum_{k=1}^N E_{\mathcal{M},k}^{\text{test}}$$



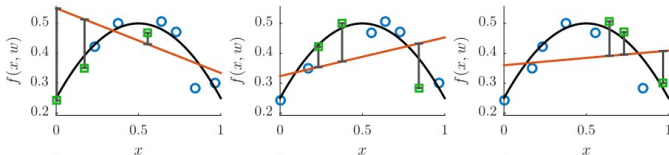
Cross-validation(1-layer)

- $K = 3$ fold cross-validation for the three linear regression models

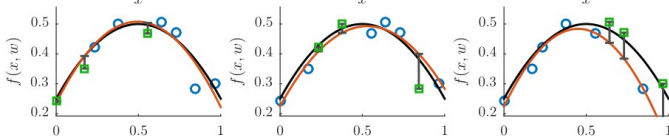
Vertically: The three models

Horizontally: The three cross-validation folds

model 1
(1st order)



model 2
(2nd order)



model 3
(6th order)

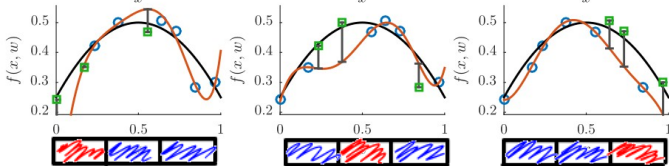
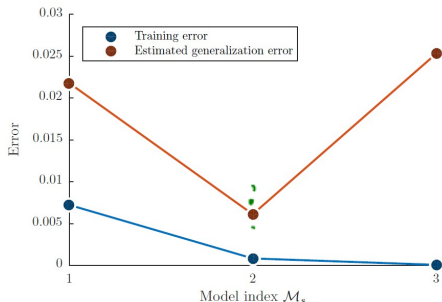


Figure 1

Cross-validation for model selection (1-layer)

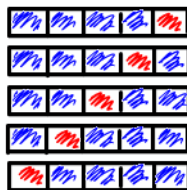
- **Purpose:** Select the best of S models
- **The idea:**
 - For each model, estimate the cross-validation error $\hat{E}_{\mathcal{M}_1}^{\text{gen}}, \dots, \hat{E}_{\mathcal{M}_S}^{\text{gen}}$ using basic cross-validation.
 - Select the optimal model $\mathcal{M}_{s^*}$ as that with the lowest error:

$$s^* = \arg \min_s \hat{E}_{\mathcal{M}_s}^{\text{gen}}$$



Cross-validation(1-layer)

- K-fold cross validation for model selection



M_1	M_2	M_3
$E_{M_1,1}^{\text{test}}$	$E_{M_2,1}^{\text{test}}$	$E_{M_3,1}^{\text{test}}$
$E_{M_1,2}^{\text{test}}$	$E_{M_2,2}^{\text{test}}$	$E_{M_3,2}^{\text{test}}$
$E_{M_1,3}^{\text{test}}$	$E_{M_2,3}^{\text{test}}$	$E_{M_3,3}^{\text{test}}$
$E_{M_1,4}^{\text{test}}$	$E_{M_2,4}^{\text{test}}$	$E_{M_3,4}^{\text{test}}$
$E_{M_1,5}^{\text{test}}$	$E_{M_2,5}^{\text{test}}$	$E_{M_3,5}^{\text{test}}$
$\hat{E}_{M_1}^{\text{gen}}$	$\hat{E}_{M_2}^{\text{gen}}$	$\hat{E}_{M_3}^{\text{gen}}$

Algorithm 4: K-fold cross-validation for model selection

Require: K , the number of folds in the cross-validation loop

Require: $M_1, \dots, M_S$. The S different models to select between

Ensure: M_{s^*} the optimal model suggested by cross-validation

for $k = 1, \dots, K$ splits do

Let $\mathcal{D}_k^{\text{train}}, \mathcal{D}_k^{\text{test}}$ the k 'th split of $\mathcal{D}$

for $s = 1, \dots, S$ models do

Train model M_s on the data $\mathcal{D}_k^{\text{train}}$

Let $E_{M_s,k}^{\text{test}}$ be the test error of the model M_s when it is tested on $\mathcal{D}_k^{\text{test}}$

end for

end for

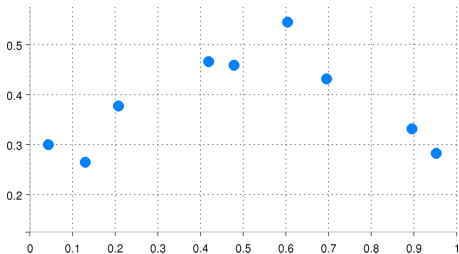
For each s compute: $\hat{E}_{M_s}^{\text{gen}} = \sum_{k=1}^K \frac{N_k^{\text{test}}}{N} E_{M_s,k}^{\text{test}}$

Select the optimal model: $s^* = \arg \min_s \hat{E}_{M_s}^{\text{gen}}$

M_{s^*} is now the optimal model suggested by cross-validation

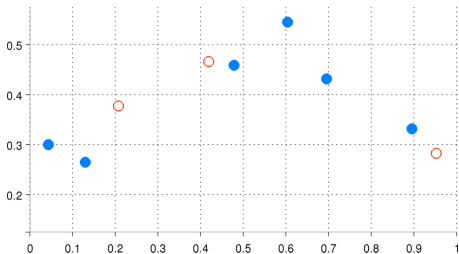
Holdout method

- Randomly choose a subset of data points to be in a **test set**
 - For example choose 1/3 of the points
- The rest is the **training set**



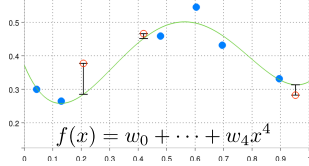
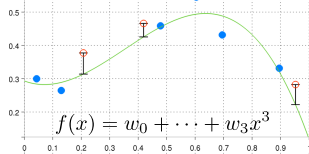
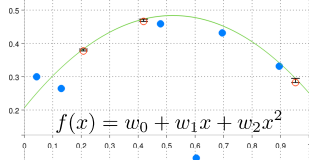
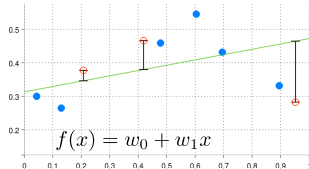
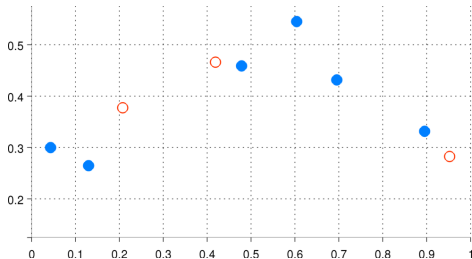
Holdout method

- Randomly choose a subset of data points to be in a **test set**
 - For example choose 1/3 of the points
- The rest is the **training set**



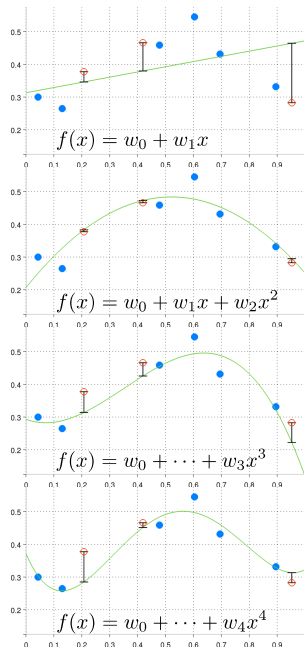
Holdout method

- Using the **training set**
 - Train the model for different complexities
- Using the **test set**
 - Compute the test error
- Choose the model with lowest **test error**



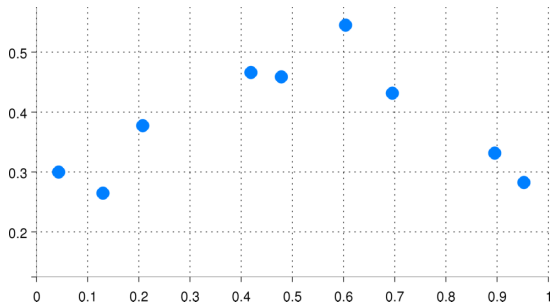
Holdout method

- Using the **training set**
 - Train the model for different complexities
- Using the **test set**
 - Compute the test error
- Choose the model with lowest **test error**



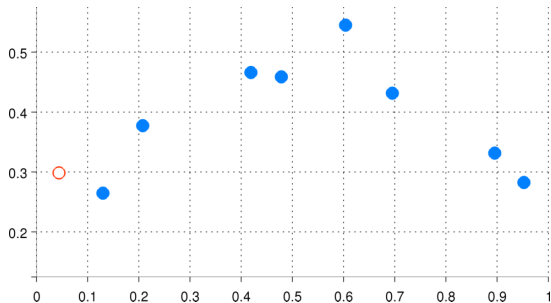
Leave-one-out

- Choose the first data point as a **test set**
- The rest is the **training set**



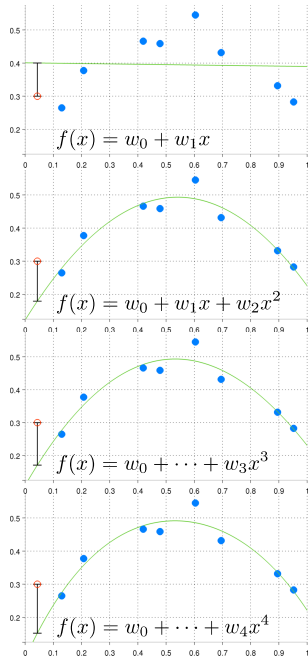
Leave-one-out

- Choose the first data point as a **test set**
- The rest is the **training set**



Leave-one-out

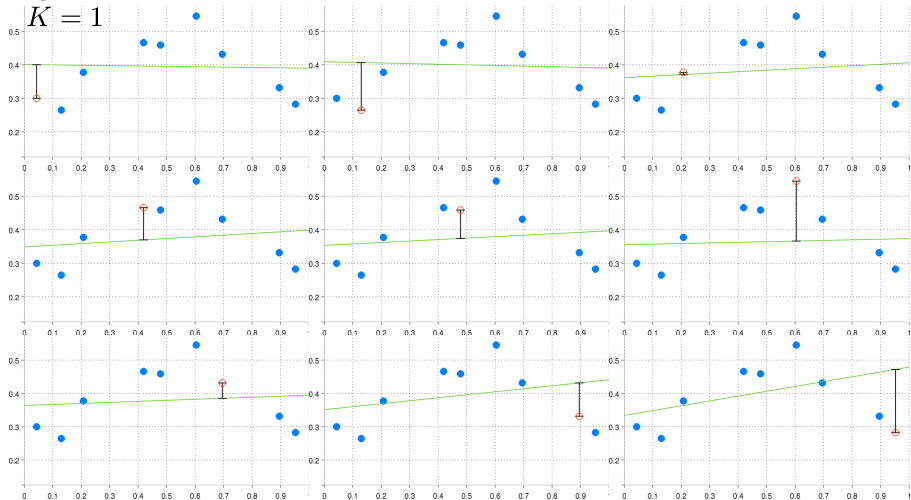
- Using the **training set**
 - Train the model for different complexities
- Using the **test set**
 - Compute the test error
- **Repeat for all data points**
 - All data points get to be test set
 - Compute **average test error**



Leave-one-out

degree in poly

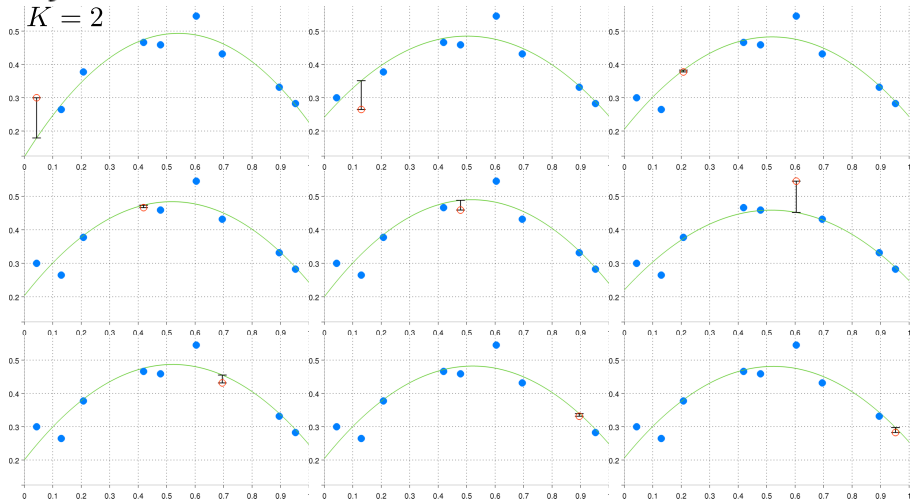
$K = 1$



Leave-one-out

poly degree

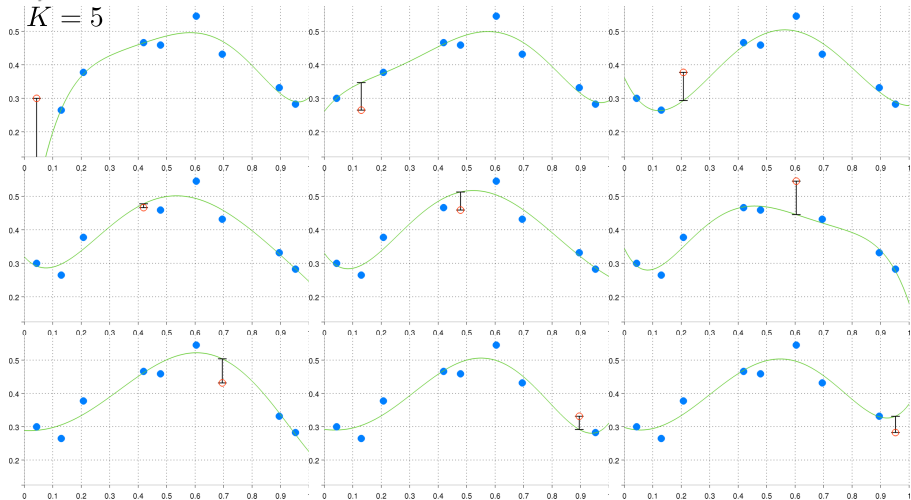
$K = 2$



Leave-one-out

poly degree

$K = 5$



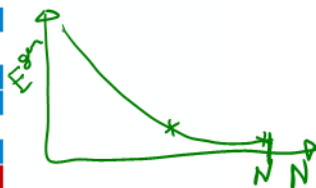
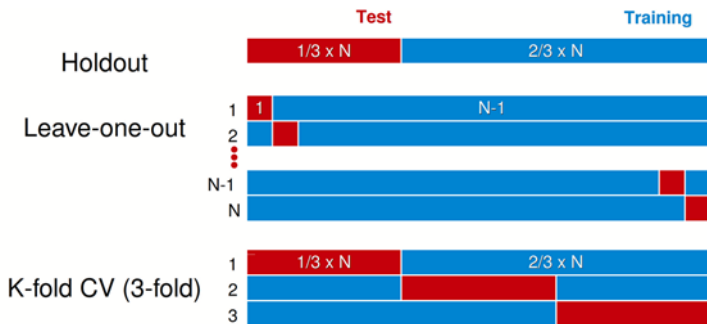
Leave-one-out

- Using the **training set**
 - Train the model for different complexities
- Using the **test set**
 - Compute the test error
- **Repeat for all data points**
 - All data points get to be test set
 - Compute **average test error**



So which method should I choose?

- Holdout is computationally least intensive, but not very data efficient
- Leave-one-out is very computationally intensive, and the estimates/models are highly correlated
- Recommendation: K -fold for $K = 5, 10$ or holdout if problem very large.



Quiz 2, Cross validation

Feature(s)	Training MSE	Test MSE
A	2.0	2.2
B	1.8	1.9
C	1.6	1.7
D	1.9	2.1
A and B	1.7	2.0
A and C	1.3	1.8
A and D	1.4	1.5
B and C	1.5	1.6
B and D	1.7	1.8
C and D	1.5	2.0
A and B and C	1.2	2.1
A and B and D	1.1	2.0
A and C and D	1.0	2.3
B and C and D	1.2	2.5
A and B and C and D	0.9	2.8

Consider a regression problem with four attributes denoted A, B, C and D.

A linear model is trained using different combinations of the attributes. The model is trained on 50% of the data and tested on the remaining 50% of the data using the hold-out method. In the table is given the training and test performance of the model for the different combinations of attributes. Which of the following statements is incorrect

- A. Hold-out 50% of the data is more computationally efficient than 5 fold cross-validation. ✓
- B. Leave one-out cross-validation gives a poor estimate of the generalization error as only one observation is part of the test set at a time. ✓
- C. The size of the training set in 10 fold cross-validation is larger than the size of the training set in 5 fold cross-validation. ✓
- D. Not all observations are used for testing using the hold-out method. ✓
- E. Don't know.

In the hold-out method a given percentage, here 50% are removed for testing and the data trained on the remaining $100\%-50\%=50\%$ of the data. This is more computationally efficient than 5 fold cross-validation as we only need to estimate one model for each combination of attributes. Leave-one-out cross-validation would however give a more precise

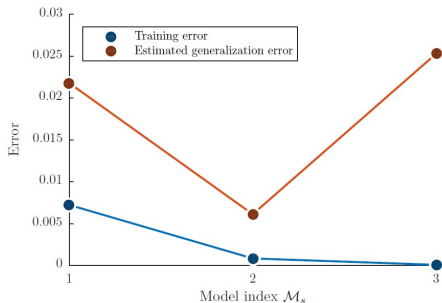
estimate of the generalization error as all observations are used in the test set once while keeping as many observations as possible for training, thus, second answer is incorrect. The size of the training set is larger in 10 fold cross-validation as 10 % of data is removed for testing whereas 20% of data is removed for testing for each fold of 5 fold cross-validation.

A problem with 1 level cross-validation?

- For each model, estimate the cross-validation error $\hat{E}_{\mathcal{M}_1}^{\text{gen}}, \dots, \hat{E}_{\mathcal{M}_S}^{\text{gen}}$ using basic cross-validation.
- Select the optimal model $\mathcal{M}_{s^*}$ as that with the lowest error:

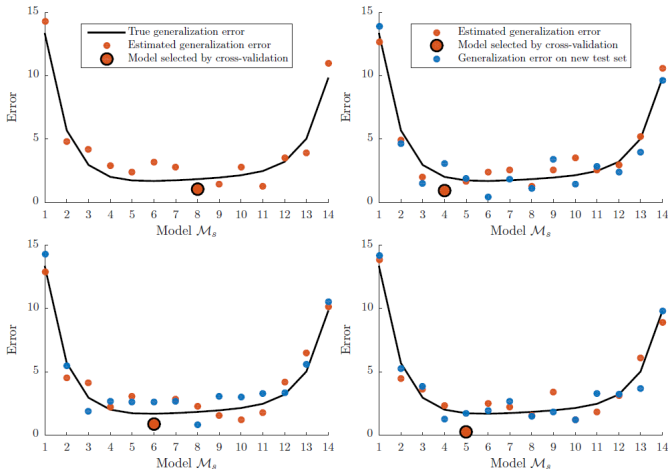
$$s^* = \arg \min_s \hat{E}_{\mathcal{M}_s}^{\text{gen}}$$

- Is the generalization error of the selected model (k=2) about 0.007?



A problem with 1 level cross-validation?

- Same as before, just with more models. Is the error of the red dot a fair estimate of the generalization error?



Two-layer cross-validation

- **Purpose:** Select optimal model and estimate generalization error of optimal model
- **How?**
 - Recall "one layer cross-validation for model selection"
 - This method returns a model (the best model)
 - We can consider "one-layer cross-validation for model selection" as a single model
- **Recall:**
 - "Basic cross-validation for model performance evaluation" estimates the generalization error of a model
- **Idea:** Apply "basic cross-validation for performance evaluation" on the "one-layer cross-validation for model selection"-model to estimate the it's generalization error



Cross-validation (2-layer)

- Two-layer cross-validation, the algorithm

Algorithm 5: Two-level cross-validation

Require: K_1, K_2 , folds in outer, and inner cross-validation loop respectively

Require: $\mathcal{M}_1, \dots, \mathcal{M}_S$: The S different models to cross-validate

Ensure: $\hat{E}^{\text{gen}}$, the estimate of the generalization error

for $i = 1, \dots, K_1$ **do**

Outer cross-validation loop. First make the outer split into K_1 folds

Let $\mathcal{D}_i^{\text{par}}, \mathcal{D}_i^{\text{test}}$ be the i 'th split of $\mathcal{D}$

for $j = 1, \dots, K_2$ **do**

Inner cross-validation loop. Use cross-validation to select optimal model

Let $\mathcal{D}_j^{\text{train}}, \mathcal{D}_j^{\text{val}}$ be the j 'th split of $\mathcal{D}_i^{\text{par}}$

for $s = 1, \dots, S$ **do**

Train $\mathcal{M}_s$ on $\mathcal{D}_j^{\text{train}}$

Let $E_{\mathcal{M}_s, j}^{\text{val}}$ be the *validation error* of the model $\mathcal{M}_s$ when it is *tested* on $\mathcal{V}_j$

end for

end for

For each s compute: $\hat{E}_s^{\text{gen}} = \sum_{j=1}^{K_2} \frac{|\mathcal{D}_j^{\text{val}}|}{|\mathcal{D}_i^{\text{par}}|} E_{\mathcal{M}_s, j}^{\text{val}}$

Select the optimal model $\mathcal{M}^* = \mathcal{M}_{s^*}$ where $s^* = \arg \min_s \hat{E}_s^{\text{gen}}$

Train $\mathcal{M}^*$ on $\mathcal{D}_i^{\text{par}}$

Let E_i^{test} be the *test error* of the model $\mathcal{M}^*$ when it is *tested* on $\mathcal{D}_i^{\text{test}}$

end for

Compute the estimate of the generalization error: $\hat{E}^{\text{gen}} = \sum_{i=1}^{K_1} \frac{|\mathcal{D}_i^{\text{test}}|}{N} E_i^{\text{test}}$

Compare models



Cross-validation (2-layer)

- Two-layer cross-validation, the algorithm

Algorithm 5: Two-level cross-validation

Require: K_1, K_2 , folds in outer, and inner cross-validation loop respectively

Require: $\mathcal{M}_1, \dots, \mathcal{M}_S$: The S different models to cross-validate

Ensure: $\hat{E}^{\text{gen}}$, the estimate of the generalization error

for $i = 1, \dots, K_1$ do

Outer cross-validation loop. First make the outer split into K_1 folds

Let $\mathcal{D}_i^{\text{par}}, \mathcal{D}_i^{\text{test}}$ be the i 'th split of $\mathcal{D}$

for $j = 1, \dots, K_2$ do

Inner cross-validation loop. Use cross-validation to select optimal model

Let $\mathcal{D}_j^{\text{train}}, \mathcal{D}_j^{\text{val}}$ be the j 'th split of $\mathcal{D}_i^{\text{par}}$

for $s = 1, \dots, S$ do

Train $\mathcal{M}_s$ on $\mathcal{D}_j^{\text{train}}$

Let $E_{\mathcal{M}_s, j}^{\text{val}}$ be the validation error of the model $\mathcal{M}_s$ when it is tested on $\mathcal{L}_j$

end for

end for

For each s compute: $\hat{E}_s^{\text{gen}} = \sum_{j=1}^{K_2} \frac{|\mathcal{D}_j^{\text{val}}|}{|\mathcal{D}_i^{\text{par}}|} E_{\mathcal{M}_s, j}^{\text{val}}$

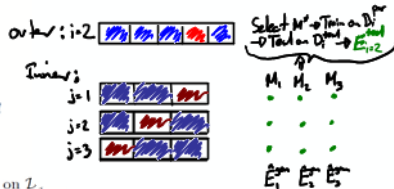
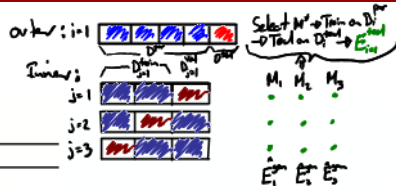
Select the optimal model $\mathcal{M}^* = \mathcal{M}_{s^*}$ where $s^* = \arg \min_s \hat{E}_s^{\text{gen}}$

Train $\mathcal{M}^*$ on $\mathcal{D}_i^{\text{par}}$

Let E_i^{test} be the test error of the model $\mathcal{M}^*$ when it is tested on $\mathcal{D}_i^{\text{test}}$

end for

Compute the estimate of the generalization error: $\hat{E}^{\text{gen}} = \sum_{i=1}^{K_1} \frac{|\mathcal{D}_i^{\text{test}}|}{N} E_i^{\text{test}}$



Compare models



Train
Test
Estimate model parameters

Quiz 3: Two-level cross-validation

Consider a classification tree model applied to a dataset of $N = 1000$ observations. Suppose we wish to both select the optimal pruning level and estimate the generalization error of the classification tree model by cross-validation. To simplify the problem, we only consider 3 possible pruning levels:

3, 4, 5.

We opt for a two-level cross-validation strategy in which we use an inner loop of $K_2 = 5$ -fold cross-validation to estimate the optimal pruning level and an outer loop of $K_1 = 10$ fold cross-validation to estimate the generalization error. That is, for each of the K_1 outer folds, the dataset is divided into

a validation set and a parameter estimation set on which K_2 -fold cross-validation is used to select the optimal pruning level for this outer fold.

How many models do we *train* using 2-level cross-validation?

- A. 50
- B. 150
- C. 160
- D. 180
- E. Don't know.

This can easily be obtained noting for each of the K_1 outer folds we must both (i) train K_2 models on the $L = 3$ different settings of pruning level (ii) train a single new model to estimate the generalization

error for this fold. Accordingly the number of trained models is

$$K_1(K_2L + 1).$$

Therefore, option C is correct.

Summary

Summary

- Decision trees
- Implications of model complexity (overfitting)
- Generalization error
 - Practical ways to estimate it (cross-validation)
 - Problem of cross-validation and how to fix it (2-layer CV)

Resources

<https://towardsdatascience.com> Alternative introduction to cross-validation

(<https://medium.com/towards-data-science/>

[cross-validation-explained-evaluating-estimator-performance-e51e5430ff85](https://medium.com/towards-data-science/cross-validation-explained-evaluating-estimator-performance-e51e5430ff85))